

Active Spiking Perception and the Slice Selection Policy

1 The Big Picture: Active Spiking Perception (ASP)

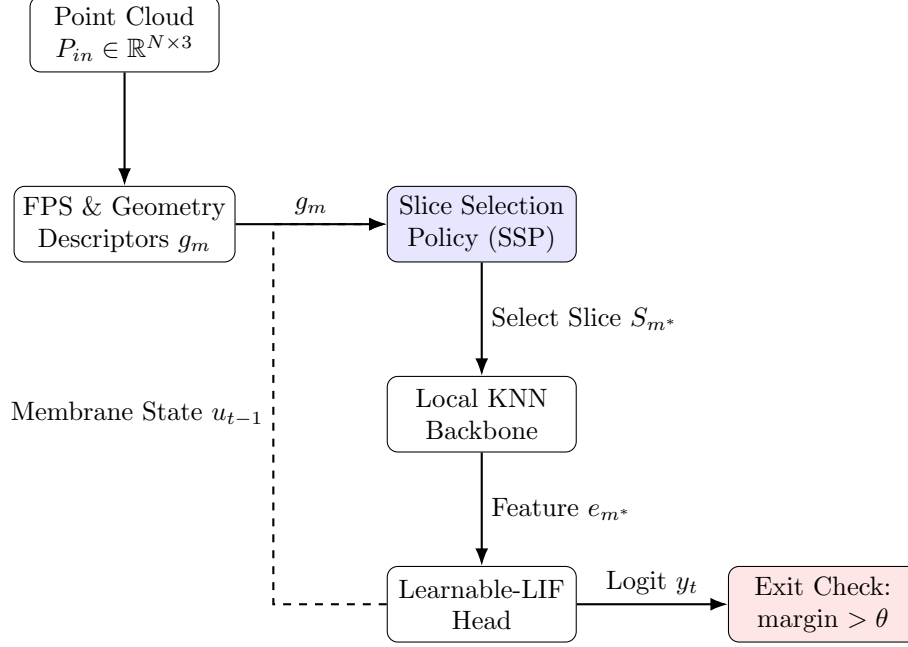
Standard Spiking Neural Network (SNN) methods process slices of a 3D point cloud in a fixed, predetermined order, regardless of the input object[cite: 30]. This wastes timesteps on regions the model is already confident about, which is energetically wasteful and computationally redundant[cite: 32].

Active Spiking Perception (ASP) fundamentally changes this by treating observation as an active, learned process. Instead of blindly processing every slice, the SNN learns which spatial region of a 3D point cloud to observe next[cite: 7]. It does this based on its current membrane potential, which acts as a running belief over the object’s class[cite: 7].

2 The Brain: Slice Selection Policy (SSP) and Architecture

The engine driving this adaptive observation is the Slice Selection Policy (SSP)[cite: 9]. The SSP is a highly lightweight cross-attention module containing roughly 2K parameters[cite: 9].

Below is the architecture diagram illustrating the inference loop where the SSP actively selects slices[cite: 90, 114].



At each step t , the SSP acts as a feedback loop, reading the current membrane state and geometry descriptors of unvisited anchors to select the most informative slice S_{m^*} [cite: 90].

2.1 The Two Inputs

To make its decision, the SSP relies on two distinct pieces of information:

- **The Membrane State (u_{t-1}):** This is the SNN’s current Leaky Integrate-and-Fire (LIF) membrane potential, acting as a compressed summary of what has been seen [cite: 33].
- **The Geometry Descriptors (g_m):** For each of the M FPS anchors, a 6-dimensional descriptor is precomputed [cite: 80]. This descriptor ($g_m \in \mathbb{R}^6$) contains the region’s centroid XYZ, distance to the cloud centroid, intra-cluster spread, and normalized point count [cite: 81, 82, 83, 84, 85].

2.2 The Scoring Mechanism

The SSP evaluates how relevant each unvisited region is by projecting the membrane state into a “key” vector and the geometry descriptors into “query” vectors [cite: 118]. It then calculates a priority score:

$$score(m) = \frac{(W_k u_{t-1})^\top (W_q g_m)}{\sqrt{d_{ssp}}} \quad (1)$$

Already visited anchors are masked to $-\infty$ to prevent redundant processing[cite: 119].

3 A Concrete Example: Classifying a Chair

For a chair, seeing the backrest first is highly discriminative[cite: 31].

Timestep $t = 0$: The SNN hasn't seen anything yet, so its membrane state is essentially empty. The SSP evaluates the geometry descriptors and selects the **Backrest** as the best starting point. The network processes the backrest, and the membrane state updates to encode a tall vertical structure.

Timestep $t = 1$: The SSP reads the updated membrane state. Knowing it needs horizontal context to confirm the shape, the SSP assigns the highest score to the **Seat**. The network processes the seat, and the membrane state updates to encode both structures, heavily implying a chair.

Timestep $t = 2$ (Early Exit): The model calculates its confidence margin $(P(top1) - P(top2))$ [cite: 111]. If this confidence margin exceeds the threshold θ , inference terminates early with the current prediction[cite: 91, 112].

4 Training vs. Inference

The ASP framework behaves differently depending on whether it is learning or being deployed:

- **During Training:** Because a hard selection is non-differentiable, the selection is made differentiable via Gumbel-softmax[cite: 10, 127]. This enables joint end-to-end training of the SSP, backbone, and temporal head[cite: 12].
- **During Inference:** A hard argmax is used to select the next slice, operating at an $\mathcal{O}(1)$ overhead per step[cite: 11].